



CHRIST (DEEMED TO BE UNIVERSITY)
BANGALORE

DEPARTMENT OF COMPUTER SCIENCE

SOFTWARE REQUIREMENTS SPECIFICATION
TENANTLY

Submitted by

Avalbir Singh Banga
2440114

B.Sc. Mathematics and Computer Science

V Semester
Academic Year 2026-2027

Project Guide: Dr. Logeshwaran J

Version 1.0 | 22 August 2026

Document Control

Field	Value
Purpose	Define the verifiable software requirements for the current Tenantly baseline and separate future roadmap items.
Primary audience	Academic reviewers, developers, testers, database engineers, product owners, and operations stakeholders.
Authority	Implemented routes, feature services, migrations, tests, README, and the implementation-ready PRD.
Baseline rule	Implemented behavior is normative. Planned requirements are labeled and do not imply delivery.
Distribution	Project team and authorized reviewers.

Revision History

Version	Date	Author	Change
1.0	22 Aug 2026	Tenantly Project Team	Initial repository-grounded SRS baseline.

Approval Record

Role	Name	Signature	Date
Project lead	<Name>	<Signature>	<Date>
Academic supervisor	<Name>	<Signature>	<Date>
Technical reviewer	<Name>	<Signature>	<Date>

Table of Contents

Document Control.....	2
Revision History	2
Approval Record.....	2
Table of Contents.....	3
1. Introduction.....	5
1.1 Purpose	5
1.2 Product Scope	5
1.3 Objectives	5
1.4 Intended Audience	5
1.5 Conventions	5
1.6 Source of Truth	5
2. Overall Description.....	6
2.1 Product Perspective.....	6
2.2 Product Functions	6
2.3 User Classes and Permissions.....	7
2.4 Operating Environment.....	7
2.5 Assumptions and Dependencies	8
2.6 Constraints and Explicit Exclusions	8
3. System Context and External Interfaces	8
3.1 Mobile User Interface	8
3.2 Authentication Interface.....	9
3.3 Database and RPC Interface	9
3.4 Storage Interface.....	9
3.5 Notification Interface.....	9
3.6 Local Device and Release Interfaces	9
4. Functional Requirements	9
4.1 Authentication, invitations, and organization context.....	9
4.2 Properties, rooms, beds, and residents	10
4.3 Tenancy and occupancy	10
4.4 Invoices, payments, allocations, and receipts	10
4.5 Complaints and maintenance tasks	11
4.6 Notices, notifications, documents, and uploads.....	12
4.7 Dashboards, reporting, expenses, and agreements.....	13
5. Data Requirements.....	13
5.1 Data Model	13

5.2 Canonical Status Models.....	14
5.3 Data Conventions.....	14
5.4 Transactional and Audit Rules.....	14
5.5 File Controls	14
6. Non-Functional Requirements	14
7. Core Use Cases and Acceptance.....	15
UC-01 Owner onboarding	15
UC-02 Monthly rent cycle	16
UC-03 Complaint resolution.....	16
UC-04 Occupancy transfer or vacancy	16
UC-05 Targeted communication.....	16
UC-06 Operational reporting	16
8. Verification and Release Acceptance	17
8.1 Verification Layers	17
8.2 Release Acceptance Criteria	17
8.3 Known Verification Notes	17
9. Requirements Traceability Matrix	17
Appendix A. Roadmap and Deferred Scope.....	21
A.1 Implemented Extensions Beyond the Original P0	21
A.2 Planned Near-Term Requirements.....	21
A.3 Explicitly Excluded Modules.....	21
Appendix B. Glossary.....	21
Appendix C. References	22
Appendix D. Requirement Status Legend.....	22

1. Introduction

1.1 Purpose

This Software Requirements Specification defines the behavioral, data, interface, security, quality, and verification requirements for Tenantly version 1.0. It is written to support academic assessment and implementation handoff. Requirements use stable identifiers so design decisions, code, database controls, and tests can be traced to a shared baseline.

1.2 Product Scope

Tenantly is an India-first property-management application for small landlords, paying-guest operators, hostel managers, tenants, and maintenance staff. It replaces fragmented spreadsheets, notebooks, and messaging threads with role-aware workflows for property inventory, residents, tenancy history, rent operations, payment evidence and approval, complaints, notices, private documents, operational expenses, and reporting. Tenantly is a private operations tool and is not a public rental marketplace.

1.3 Objectives

- Allow a first-time owner to configure an organization, property, room, resident, and tenancy without leaving the application.
- Preserve auditable occupancy, financial, complaint, and decision history.
- Enforce organization, property, and resident isolation at the database and storage layers.
- Provide focused mobile workflows for owners/managers, tenants, and maintenance staff.
- Favor reliable manual payment verification while keeping the data model extensible.

1.4 Intended Audience

Audience	How to use this document
Product and academic reviewers	Evaluate scope, objectives, completeness, constraints, and traceability.
Mobile developers	Implement and review role-aware screens, validation, state, and error behavior.
Database engineers	Verify schema, functions, constraints, RLS, storage policies, and transactional invariants.
Test engineers	Derive unit, integration, database, accessibility, and end-to-end scenarios.
Operators	Confirm that workflows match practical property-management responsibilities.

1.5 Conventions

- Shall identifies a mandatory requirement; should identifies a recommended quality target.
- Implemented means repository evidence supports the behavior; partially implemented means a dependency or verification gap remains.
- Planned items are non-baseline roadmap requirements; excluded items are intentionally outside this release.
- Money values are expressed to users in INR and stored internally as integer paise.
- Dates in this document use day-month-year wording to avoid numeric ambiguity.

1.6 Source of Truth

The current baseline is derived from the application routes and feature services, generated database types, Supabase migrations and policies, project tests, README, demonstration guide, and implementation-ready product requirements. When prose and implementation differ, the implemented schema and guarded behavior define current status; the difference is recorded as planned or partial.

2. Overall Description

2.1 Product Perspective

Tenantly is a cross-platform Expo/React Native client backed exclusively by Supabase. Supabase Auth owns identity; PostgreSQL stores organization-scoped business records; Row Level Security enforces access; private Storage holds evidence and documents; PostgreSQL functions execute multi-row financial and occupancy operations; and Expo Notifications supports push delivery when an EAS project is configured.

Tenantly system architecture



Figure 1. Tenantly system architecture.

2.2 Product Functions

- Account registration, recovery, invitation acceptance, membership context, and role-aware navigation.
- Property, room, bed, resident, tenancy, transfer, vacancy, and occupancy-history management.
- Monthly invoice generation, manual payment evidence, decision, allocation, balance, and receipt workflows.
- Tenant complaints, owner triage, maintenance assignment, event history, and reopening.
- Targeted notices, read state, private documents, in-app notifications, and optional push delivery.
- Owner and tenant dashboards, CSV exports, receipt PDFs, expenses, profit summaries, and agreement renewal.

Role-aware use-case model

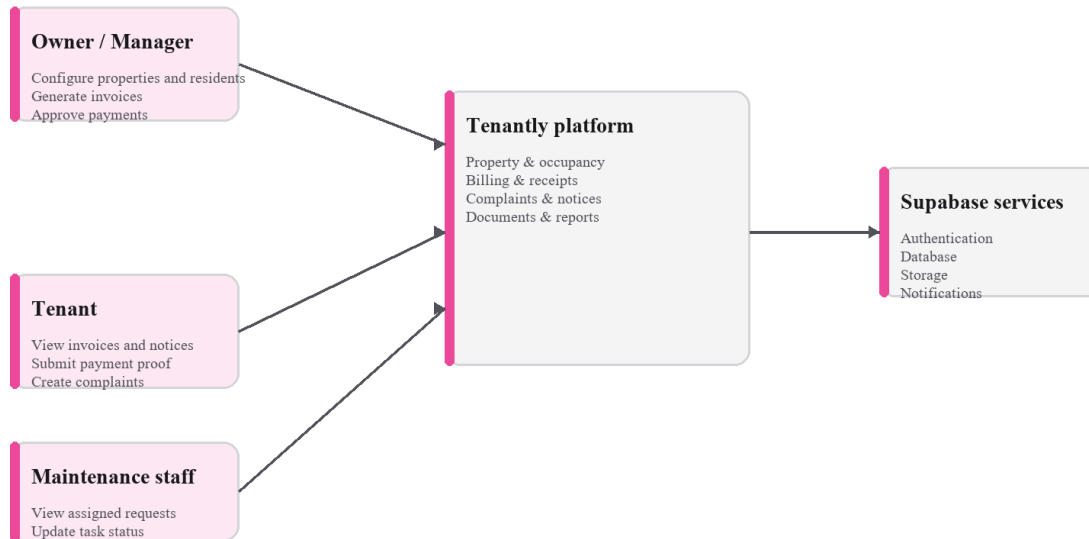


Figure 2. Role-aware use-case model.

2.3 User Classes and Permissions

Role	Scope	Principal permissions	Restrictions
Owner	Organization	Full organization, property, resident, finance, member, notice, complaint, document, expense, and report operations.	Cannot bypass server validation or access another organization.
Manager	Organization/property	Operational access limited by membership and assigned properties.	Cannot assume ownership or platform privileges; finance actions require explicit authorization.
Tenant	Linked resident/tenancy	Own invoices, payments, receipts, documents, complaints, and targeted notices.	Cannot see other residents or decide payments.
Maintenance staff	Assigned properties/tasks	View assigned complaints, add notes, and perform allowed task transitions.	No general finance, membership, or unrelated resident access.
Platform administrator	Trusted external platform	Future support and platform operations outside the mobile client.	Never granted by an organization or mobile route.

2.4 Operating Environment

Layer	Baseline
Client	Expo SDK 56, React Native 0.85, React 19, TypeScript strict mode, Expo Router, Hermes.
Platforms	Android 7 or later; iOS 16.4 or later; web export as a fallback demonstration surface.
State and forms	TanStack Query, Zustand for limited UI context, React Hook Form, and Zod.
Backend	Supabase Auth, PostgreSQL, RLS, private Storage, database functions, and targeted notifications.
Build/release	npm, Expo tooling, EAS profiles, compatible migrations, and optional Sentry configuration.

2.5 Assumptions and Dependencies

- A network connection is required for authoritative financial, occupancy, invitation, and receipt operations.
- Supabase migrations are applied in timestamp order and remain compatible with the deployed client.
- The client receives only the Supabase project URL and publishable key; privileged secrets are server-side.
- Push delivery depends on a configured EAS project and device permission; in-app notifications remain authoritative.
- English is the first interface language; user-facing strings remain centralized for later localization.
- Default property timezone is Asia/Kolkata, while stored timestamps remain UTC.

2.6 Constraints and Explicit Exclusions

- No online payment gateway, subscription billing, WhatsApp/SMS integration, public marketplace, maps, or booking workflow is part of version 1.0.
- No platform-administrator mobile console is included.
- No full offline synchronization or offline financial mutation queue is included.
- No automated legal agreement generation, advanced forecasting, visitor/security desk, meter, or mess-management module is included.

3. System Context and External Interfaces

3.1 Mobile User Interface

The client uses role-specific Expo Router groups with shared authentication, profile, organization selection, notifications, configuration-required, unauthorized, and not-found states. Owners and managers receive property and finance operations; tenants receive personal financial, notice, document, and request workflows; maintenance staff receive assigned task workflows. Unauthorized deep links shall show a safe error without confirming protected record existence.

Role-aware navigation

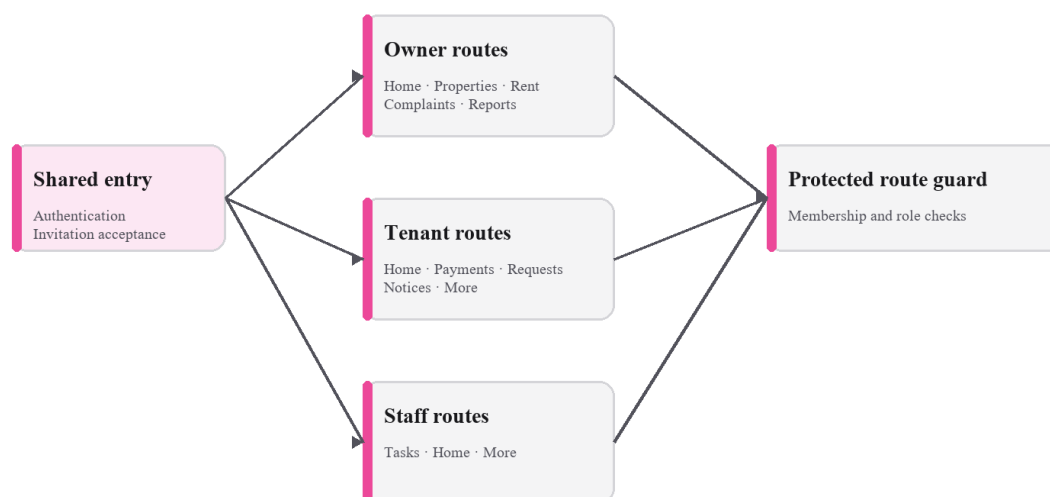


Figure 3. Role-aware navigation model.

3.2 Authentication Interface

The client integrates with Supabase Auth using the project URL and publishable key. Session restoration and token refresh are owned by the authentication provider. Recovery and invitation callbacks use deep links. Privileged service-role credentials are prohibited from the mobile bundle.

3.3 Database and RPC Interface

Standard reads and low-risk writes may use the Supabase client when RLS and constraints fully protect the operation. Multi-row occupancy, invitation, billing, payment, complaint, notice, expense, and renewal changes use authenticated PostgreSQL functions. Client Zod validation improves usability; database checks remain authoritative.

3.4 Storage Interface

Private Storage buckets hold payment proof, complaint images, resident documents, agreements, property media, and expense receipts. Business rows store object paths, not public URLs. Access policies mirror the owning organization, resident, complaint, or expense row, and authorized retrieval uses time-limited signed access.

3.5 Notification Interface

Database events create in-app notification records for payment decisions, notices, and complaint activity. Expo Notifications registers supported devices when EAS configuration is present. Device registration or push-delivery failure must not prevent the underlying business operation or in-app record.

3.6 Local Device and Release Interfaces

SecureStore retains session material. Image and document pickers collect evidence; image manipulation prepares proof images; print and sharing APIs export receipts. EAS Build, Submit, and Update provide releases, with native dependency changes requiring a new binary and schema changes requiring compatible migration deployment.

4. Functional Requirements

Each requirement is normative for the stated status. Acceptance text defines the observable outcome required for verification. Current-status labels reflect repository evidence as of 22 August 2026.

4.1 Authentication, invitations, and organization context

ID	Status	Requirement	Acceptance criterion
FR-AUTH-001	Implemented	The system shall support email/password registration, email verification, login, logout, forgotten-password initiation, and password reset.	A valid user can complete each authentication flow; invalid or expired credentials return safe, actionable feedback.
FR-AUTH-002	Implemented	The system shall restore a valid authenticated session when the application restarts.	A returning authenticated user reaches the correct protected route without entering credentials again.
FR-AUTH-003	Implemented	An authenticated user without a membership shall be routed to organization setup or invitation acceptance.	No organization-scoped data is requested before a membership is active.
FR-AUTH-004	Implemented	An owner shall be able to create an organization and initial owner membership atomically.	One organization and one active owner membership are created; repeated submission cannot create duplicates.
FR-AUTH-005	Implemented	An owner shall be able to issue an expiring invitation for an approved organization role and optional property scope.	The invitation records role, organization, expiry, and permitted properties and rejects malformed email addresses.
FR-AUTH-006	Implemented	An invited user shall be able to accept a valid invitation and obtain the intended membership.	Acceptance is idempotent, rejects expired/revoked tokens, and never grants a broader role or property scope.

ID	Status	Requirement	Acceptance criterion
FR-AUTH-007	Implemented	Users with multiple memberships shall select an active organization context.	Queries and navigation change to the selected organization while server authorization remains membership-based.
FR-AUTH-008	External	Platform-administrator privileges shall not be assignable through the mobile client.	No client route, form, or organization function can grant platform-level privileges.

4.2 Properties, rooms, beds, and residents

ID	Status	Requirement	Acceptance criterion
FR-PROP-001	Implemented	An authorized owner or manager shall list accessible properties and view their operational status.	Only properties permitted by organization and property scope are returned.
FR-PROP-002	Implemented	An owner shall create and edit a property with name, type, address, timezone, and status.	Required fields are validated and the row is assigned to the active organization.
FR-PROP-003	Implemented	An owner shall archive an unoccupied property while preserving history.	Archiving is rejected while an active occupancy exists and historical records remain queryable.
FR-PROP-004	Implemented	An authorized operator shall create, edit, list, and archive rooms under an accessible property.	Room identifiers remain unique within the property and occupied rooms cannot be archived.
FR-PROP-005	Implemented	Shared rooms shall support optional bed inventory.	Beds are created under one room, carry stable status values, and can be referenced by occupancy assignments.
FR-PROP-006	Implemented	An owner shall create a resident record before or after the resident has an application account.	The resident may initially have no profile link and can later be associated through an invitation.
FR-PROP-007	Implemented	An authorized operator shall view and update resident contact, identity, status, and tenancy-related information.	Validation errors do not partially save the resident and unauthorized profiles remain inaccessible.
FR-PROP-008	Implemented	Resident and inventory deletion shall use archival states when history is material.	Historical tenancy, occupancy, billing, and audit references remain intact.

4.3 Tenancy and occupancy

ID	Status	Requirement	Acceptance criterion
FR-OCC-001	Implemented	An authorized operator shall create a tenancy with start date, rent, deposit, due day, currency, and initial assignment.	Tenancy and occupancy assignment are committed atomically or not at all.
FR-OCC-002	Implemented	The system shall preserve dated occupancy-assignment history.	Each transfer or vacancy closes the prior active period rather than overwriting it.
FR-OCC-003	Implemented	The system shall prevent overlapping active assignments to an exclusive bed.	A conflicting assignment is rejected by server-side rules.
FR-OCC-004	Implemented	An authorized operator shall transfer a resident to another room or bed atomically.	The old assignment ends and the new assignment begins in one transaction.
FR-OCC-005	Implemented	An authorized operator shall vacate a resident using an effective date.	The active assignment is closed, tenancy status is updated consistently, and prior occupancy remains visible.
FR-OCC-006	Implemented	Vacancy and occupancy metrics shall be derived from active assignments.	Displayed availability agrees with current assignment records and is not maintained as an editable counter.
FR-OCC-007	Implemented	A tenant shall access only the tenancy and occupancy records linked to the tenant's resident profile.	Requests for another resident's records return no protected data.

4.4 Invoices, payments, allocations, and receipts

ID	Status	Requirement	Acceptance criterion
FR-BILL-001	Implemented	An owner or authorized manager shall generate monthly invoices for eligible active tenancies.	The operation returns created, skipped, and failed outcomes for the selected billing month.
FR-BILL-002	Implemented	Monthly invoice generation shall be idempotent.	Repeating generation for the same tenancy and billing period does not create a duplicate invoice.

ID	Status	Requirement	Acceptance criterion
FR-BILL-003	Implemented	Invoices shall snapshot applicable tenancy terms and support rent, utilities, mess, discount, late-fee, and adjustment items.	Totals are derived from integer-paise line items and do not depend on later tenancy edits.
FR-BILL-004	Implemented	The system shall represent draft, issued, partially paid, paid, overdue, waived, and void invoice states.	Only valid status values are stored and displayed consistently.
FR-BILL-005	Implemented	A tenant shall submit UPI, bank-transfer, or other manual payment proof with amount, date, reference, and optional evidence image.	The payment enters submitted state and cannot be self-approved by the tenant.
FR-BILL-006	Implemented	An authorized operator shall record a cash or other manual payment for a resident.	The payment is created within the correct organization and payer scope.
FR-BILL-007	Implemented	An authorized owner or manager shall approve or reject a submitted payment; rejection shall require a reason.	The decision is atomic, audited, and visible to the payer.
FR-BILL-008	Implemented	Approved payments shall support one or more allocations without exceeding the approved amount or invoice balance.	Invalid over-allocation is rejected server-side and balances reconcile after partial payments.
FR-BILL-009	Implemented	Approval shall generate an immutable, uniquely numbered receipt when allocation conditions are met.	Receipt amount equals approved allocations and update/delete attempts are rejected.
FR-BILL-010	Implemented	Authorized users shall view invoice ledgers, payment decisions, receipts, and outstanding balances.	Each role sees only its permitted organization or resident records.
FR-BILL-011	Implemented	The tenant shall be able to export or share an authorized receipt as PDF.	The generated receipt contains the correct receipt number, payment, allocation, and payer details.

Monthly invoice and payment workflow

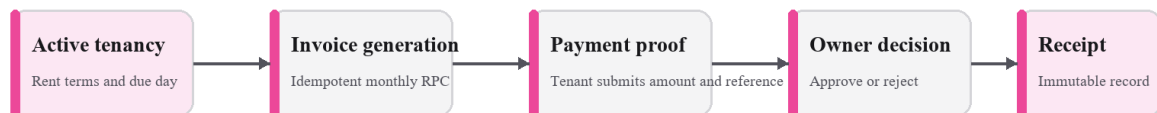


Figure 4. Monthly invoice and payment workflow.

4.5 Complaints and maintenance tasks

ID	Status	Requirement	Acceptance criterion
FR-CMP-001	Implemented	A linked tenant shall create a complaint with category, priority, description, and optional images.	The complaint is associated with the tenant's resident, property, and organization records.
FR-CMP-002	Implemented	An authorized operator shall triage, assign, and transition a complaint through the allowed lifecycle.	Invalid transitions are rejected by the server and each accepted transition produces an event.

ID	Status	Requirement	Acceptance criterion
FR-CMP-003	Implemented	Maintenance staff shall see only complaints assigned within their authorized property scope.	Unassigned or out-of-scope requests are not returned.
FR-CMP-004	Implemented	Owners, managers, and assigned staff shall add progress notes to a complaint.	Each note records actor, time, status context, and complaint identifier.
FR-CMP-005	Implemented	A tenant shall view the complaint event history and reopen an eligible resolved complaint.	Reopening adds a new event and preserves prior resolution history.
FR-CMP-006	Implemented	Complaint attachments shall be stored privately and accessed through authorized signed requests.	A user outside the permitted organization, resident, or assignment scope cannot retrieve the attachment.

Complaint resolution workflow

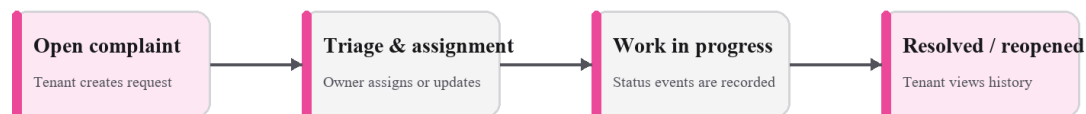


Figure 5. Complaint resolution workflow.

4.6 Notices, notifications, documents, and uploads

ID	Status	Requirement	Acceptance criterion
FR-COM-001	Implemented	An authorized operator shall publish a notice to an organization, property, room, or resident target.	Target validation prevents cross-organization references and preserves pinned state.
FR-COM-002	Implemented	A member shall see only notices whose targets include that member's authorized context.	Non-targeted notices are not returned even when their identifiers are known.
FR-COM-003	Implemented	A member shall mark an accessible notice as read.	The read state is stored per profile and cannot be written for another profile.
FR-COM-004	Implemented	The system shall create in-app notifications for payment decisions, notices, and complaint updates.	Notifications contain recipient scope, read state, and a safe application destination.
FR-COM-005	Partially implemented	The system shall register push devices and deliver configured push notifications for supported events.	Registration activates only with a configured EAS project; failure does not block in-app notifications.
FR-COM-006	Implemented	Authorized users shall upload and view resident IDs, agreements, payment proof, complaint media, and expense receipts subject to file controls.	Files remain in private buckets; image uploads are at most 10 MB and documents at most 15 MB.

4.7 Dashboards, reporting, expenses, and agreements

ID	Status	Requirement	Acceptance criterion
FR-OPS-001	Implemented	The owner dashboard shall show occupancy, expected rent, collected rent, outstanding rent, pending approvals, complaints, and profit indicators.	Values are returned from organization-scoped queries or database functions and agree with source records.
FR-OPS-002	Implemented	The tenant dashboard shall show current due amount, recent payment state, active complaints, and pinned notices.	Every displayed record is linked to the authenticated tenant.
FR-OPS-003	Implemented	Authorized users shall export invoice and payment list data as CSV.	The export uses consistent columns, INR values, dates, and organization-scoped records.
FR-OPS-004	Implemented	An owner or authorized manager shall record and void operational expenses with category, amount, date, description, and optional receipt.	Expenses use integer paise, remain auditable, and voiding does not silently delete history.
FR-OPS-005	Implemented	The system shall report monthly collected amount, expense amount, and net profit and shall support agreement renewal with expiry tracking.	Report totals reconcile to scoped records, and renewal preserves the prior document history.

5. Data Requirements

5.1 Data Model

Core data model

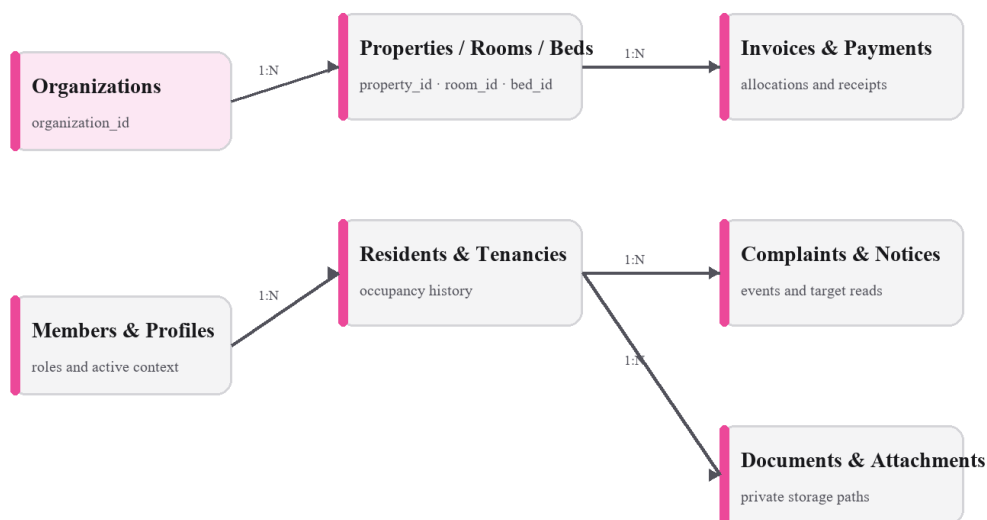


Figure 6. Core data model.

Domain	Principal entities	Required invariants
Identity and access	profiles, organizations, organization_memberships, property_memberships, invitations, invitation_properties	Memberships carry organization roles; property grants narrow access; invitation acceptance cannot broaden scope.
Property and occupancy	properties, property_media, rooms, beds, residents, tenancies, occupancy_assignments	Every business row is organization-scoped; active exclusive occupancy cannot overlap; history is dated and preserved.
Billing	invoices, invoice_items, payments, payment_allocations, receipts	Money uses paise; generation is idempotent; allocations reconcile; receipts are immutable and uniquely numbered.
Operations	complaints, complaint_events, notices, notice_targets, notice_reads, expenses	Transitions and targets are validated; events and voids preserve history.

Domain	Principal entities	Required invariants
Files and communication	attachments, documents, notifications, push_devices	Only object paths are stored; recipient and business-row scope control access.
Assurance	audit_logs	Sensitive decisions record actor, event, entity, and timestamp without exposing secret payloads.

5.2 Canonical Status Models

Model	Allowed values
Membership	invited, active, suspended, revoked
Property	active, archived
Tenancy	draft, active, notice_period, ended, cancelled
Invoice	draft, issued, partially_paid, paid, overdue, waived, void
Payment	submitted, approved, rejected, partially_refunded, refunded
Complaint	open, assigned, in_progress, resolved, closed, reopened, rejected
Expense	recorded, void

5.3 Data Conventions

- Primary keys are UUID values; profiles.id corresponds to auth.users.id.
- Business tables include organization_id and created/updated timestamps where applicable.
- Instants use timestamptz; local calendar values use date; money uses bigint paise plus currency code.
- Foreign keys, unique constraints, enums/checks, triggers, RLS, and transactional functions enforce integrity.
- Operational records are archived rather than deleted when historical meaning exists.

5.4 Transactional and Audit Rules

- Organization creation, tenancy plus initial assignment, occupancy transfer, invoice generation, payment decision/allocation, complaint transition, invitation acceptance, notice publication, expense operations, and agreement renewal shall execute through protected server operations.
- Receipt records cannot be updated or deleted after creation.
- Approvals, rejections, reversals, transitions, and voids retain actor and timestamp evidence.
- Cross-organization identifiers and inconsistent parent-child relationships are rejected before commit.

5.5 File Controls

Category	Accepted types	Maximum	Storage behavior
Images	JPEG, PNG, HEIC	10 MB each	Prepare/compress for display while retaining readable evidence; store privately.
Documents	PDF, JPEG, PNG	15 MB each	Validate ownership and store a protected object path.

6. Non-Functional Requirements

ID	Status	Requirement	Verification criterion
NFR-SEC-001	Implemented	Authorization shall be enforced with PostgreSQL Row Level Security on every exposed business table, not solely through client navigation.	Negative role and cross-organization access tests return no protected data.
NFR-SEC-002	Implemented	The mobile bundle shall contain only public configuration such as the Supabase URL and publishable key; service-role and provider secrets shall remain server-side.	Build configuration and repository scanning reveal no privileged credential in EXPO_PUBLIC values.

ID	Status	Requirement	Verification criterion
NFR-SEC-003	Implemented	Session and refresh material shall use platform secure storage.	Authentication persistence uses SecureStore rather than plain application storage.
NFR-PRV-001	Implemented	Sensitive files shall use private buckets and short-lived authorized access rather than permanent public URLs.	Anonymous and unauthorized retrieval attempts fail.
NFR-PRV-002	Partially implemented	The system shall validate file type, size, ownership, and storage path and should remove unnecessary image metadata where practical.	Client limits and storage policies are enforced; metadata stripping is verified for supported image workflows.
NFR-DAT-001	Implemented	Money shall be stored as integer paise with a three-letter currency code; floating-point storage is prohibited.	Schema types and calculations preserve exact totals for partial allocations and reports.
NFR-DAT-002	Implemented	Timestamps shall be stored as UTC timestampz, calendar dates as date, and property presentation shall default to Asia/Kolkata unless configured otherwise.	Database types and UI formatting produce the intended local date without altering stored instants.
NFR-DAT-003	Implemented	Financial, occupancy, receipt, and audit history shall be immutable or changed through explicit state transitions or compensating records.	Direct destructive changes are blocked where required and history remains queryable.
NFR-REL-001	Implemented	Multi-row financial and occupancy operations shall be transactional and idempotent where retry is expected.	Failure leaves no partial state and repeated requests do not duplicate protected records.
NFR-REL-002	Implemented	Missing backend configuration shall show a configuration-required state and shall never substitute mock production data.	Launching without required public variables displays the safe configuration screen.
NFR-ERR-001	Implemented	User-facing failures shall explain what happened and provide a recovery action without exposing sensitive payloads.	Representative service, validation, network, and authorization failures map to safe application messages.
NFR-OFF-001	Partially implemented	The P0 client shall fail gracefully when offline; financial approvals, invoice generation, assignments, and receipt creation shall remain online-only.	Offline state is visible and protected mutations do not claim success without server confirmation.
NFR-ACC-001	Implemented	Interactive targets shall be at least 44 pt on iOS and 48 dp on Android, with accessible names, states, and non-gesture alternatives.	Manual accessibility review confirms target size, labeling, and keyboard/screen-reader operation.
NFR-ACC-002	Partially implemented	Layouts shall support large text, sufficient contrast, logical screen-reader order, reduced motion, and reduced transparency without clipping essential values.	Manual checks pass on representative owner, tenant, and staff screens.
NFR-CMP-001	Implemented	The application shall support the Expo SDK 56 baseline: Android 7 or later and iOS 16.4 or later.	Supported-platform builds install and complete core journeys.
NFR-MNT-001	Implemented	The TypeScript client shall use strict typing, generated database types, feature-oriented modules, scoped query keys, and centralized user-facing strings.	Typecheck, code review, and project-structure inspection meet the stated conventions.
NFR-PER-001	Partially implemented	Potentially unbounded lists shall be paginated and dashboard data shall use scoped aggregate queries rather than full-table client calculations.	Large seeded datasets render without downloading unrelated organization records.
NFR-OBS-001	Partially implemented	The production build shall support crash monitoring without logging secrets or sensitive payloads.	Sentry activates only when configured and test events exclude protected data.
NFR-REL-003	Partially implemented	A production release shall pass typecheck, lint, formatting, client tests, database reset/tests, Expo Doctor, and representative cross-role smoke tests.	Release evidence records every gate; environment-blocked checks are not reported as passing.

7. Core Use Cases and Acceptance

UC-01 Owner onboarding

Primary actor: Owner

1. Register and verify email.
2. Create an organization.
3. Create a property and room/bed inventory.
4. Create a resident and tenancy with initial assignment.
5. Issue a tenant invitation.

Acceptance: The flow completes in-app, survives safe restart, and duplicate submission creates no duplicate organization, tenancy, or assignment.

UC-02 Monthly rent cycle

Primary actor: Owner/manager and tenant

1. Select a property and billing month.
2. Preview eligible active tenancies.
3. Generate missing invoices.
4. Tenant opens an invoice and submits proof or operator records cash.
5. Authorized operator approves/rejects and allocations update balances.
6. Tenant views or shares the resulting receipt.

Acceptance: Generation is idempotent; payment decision is atomic; partial balances reconcile; receipts are immutable.

UC-03 Complaint resolution

Primary actor: Tenant, owner/manager, maintenance staff

1. Tenant creates a categorized complaint.
2. Operator triages and assigns it.
3. Assigned staff records progress and allowed transitions.
4. Tenant views history and may reopen an eligible resolution.

Acceptance: Every accepted state change produces a timestamped event and unauthorized staff cannot see the request.

UC-04 Occupancy transfer or vacancy

Primary actor: Owner/manager

1. Open an active tenancy.
2. Choose a new room/bed or vacancy date.
3. Confirm the protected operation.
4. Review updated occupancy and vacancy metrics.

Acceptance: The prior period closes, the new state starts atomically, conflicts are rejected, and history remains queryable.

UC-05 Targeted communication

Primary actor: Owner/manager and member

1. Compose a notice and select an organization/property/room/resident target.
2. Publish the notice.
3. Eligible members receive an in-app notification and read the notice.
4. The member marks it as read.

Acceptance: Only intended recipients can retrieve the notice and each read state belongs to the authenticated profile.

UC-06 Operational reporting

Primary actor: Owner/manager

1. Record categorized expenses and optional receipts.
2. Review collected rent, expenses, and net profit for a selected month.
3. Export scoped billing data and renew an expiring agreement when required.

Acceptance: Totals reconcile to source records, exports contain the expected columns, and void/renewal actions preserve history.

8. Verification and Release Acceptance

8.1 Verification Layers

Layer	Coverage
Static analysis	TypeScript strict typecheck, ESLint, formatting, generated database-type compatibility, and Expo Doctor.
Unit/component	Money, dates, CSV, query keys, errors, validation, services, aggregates, state views, and navigation.
Database	Schema, constraints, functions, RLS, storage policies, organization isolation, and release-readiness using pgTAP.
End-to-end	Owner setup, monthly rent, payment proof/decision, complaint lifecycle, staff tasks, notices, and occupancy transfer.
Manual quality	Accessibility, supported-device behavior, offline failure states, private-file handling, and visual review.

8.2 Release Acceptance Criteria

- An owner completes setup and invites a tenant without leaving the application.
- Occupancy history survives transfer and vacancy and prevents exclusive-bed conflicts.
- Invoice generation is idempotent; payment decisions, allocations, balances, and receipts reconcile.
- Owner, manager, tenant, and staff data access is isolated by passing positive and negative RLS tests.
- Complaints, targeted notices, documents, dashboards, expenses, and agreement renewal work end to end.
- Required static, client, database, accessibility, and cross-role checks are recorded; blocked checks are not described as passing.

8.3 Known Verification Notes

The repository contains Jest and pgTAP suites. A prior project-report run recorded passing client tests, typecheck, and lint, while local database execution was environment-blocked because the Supabase/PostgreSQL stack was not running. This SRS treats database test availability as evidence of planned verification, not as proof of a current pass. Expo Doctor also recorded an SDK/Hermes advisory that should be resolved during dependency maintenance.

9. Requirements Traceability Matrix

The matrix maps every normative requirement to representative repository evidence and its principal verification method. Evidence identifies implementation surfaces, not proof that every runtime test has passed in the current environment.

Requirement	Status	Representative evidence	Verification method
FR-AUTH-001	Implemented	Auth routes; auth service/provider; organization and invitation services; invitation RPCs and RLS	Unit/service tests, authenticated route checks, pgTAP RLS tests, manual invitation flow
FR-AUTH-002	Implemented	Auth routes; auth service/provider; organization and invitation services; invitation RPCs and RLS	Unit/service tests, authenticated route checks, pgTAP RLS tests, manual invitation flow
FR-AUTH-003	Implemented	Auth routes; auth service/provider; organization and invitation services; invitation RPCs and RLS	Unit/service tests, authenticated route checks, pgTAP RLS tests, manual invitation flow
FR-AUTH-004	Implemented	Auth routes; auth service/provider; organization and invitation services; invitation RPCs and RLS	Unit/service tests, authenticated route checks, pgTAP RLS tests, manual invitation flow
FR-AUTH-005	Implemented	Auth routes; auth service/provider; organization and invitation services; invitation RPCs and RLS	Unit/service tests, authenticated route checks, pgTAP RLS tests, manual invitation flow
FR-AUTH-006	Implemented	Auth routes; auth service/provider; organization and invitation services; invitation RPCs and RLS	Unit/service tests, authenticated route checks, pgTAP RLS tests, manual invitation flow

Requirement	Status	Representative evidence	Verification method
FR-AUTH-007	Implemented	Auth routes; auth service/provider; organization and invitation services; invitation RPCs and RLS	Unit/service tests, authenticated route checks, pgTAP RLS tests, manual invitation flow
FR-AUTH-008	External	Auth routes; auth service/provider; organization and invitation services; invitation RPCs and RLS	Unit/service tests, authenticated route checks, pgTAP RLS tests, manual invitation flow
FR-PROP-001	Implemented	Owner property/resident routes; property, room, and resident services; property and inventory schema/RLS	Service tests, database constraints, owner/manager/tenant access tests
FR-PROP-002	Implemented	Owner property/resident routes; property, room, and resident services; property and inventory schema/RLS	Service tests, database constraints, owner/manager/tenant access tests
FR-PROP-003	Implemented	Owner property/resident routes; property, room, and resident services; property and inventory schema/RLS	Service tests, database constraints, owner/manager/tenant access tests
FR-PROP-004	Implemented	Owner property/resident routes; property, room, and resident services; property and inventory schema/RLS	Service tests, database constraints, owner/manager/tenant access tests
FR-PROP-005	Implemented	Owner property/resident routes; property, room, and resident services; property and inventory schema/RLS	Service tests, database constraints, owner/manager/tenant access tests
FR-PROP-006	Implemented	Owner property/resident routes; property, room, and resident services; property and inventory schema/RLS	Service tests, database constraints, owner/manager/tenant access tests
FR-PROP-007	Implemented	Owner property/resident routes; property, room, and resident services; property and inventory schema/RLS	Service tests, database constraints, owner/manager/tenant access tests
FR-PROP-008	Implemented	Owner property/resident routes; property, room, and resident services; property and inventory schema/RLS	Service tests, database constraints, owner/manager/tenant access tests
FR-OCC-001	Implemented	Tenancy and occupancy routes/services; create_tenancy_with_assignment and transfer_occupancy RPCs	Transaction, overlap, transfer, vacancy, and cross-tenant pgTAP scenarios
FR-OCC-002	Implemented	Tenancy and occupancy routes/services; create_tenancy_with_assignment and transfer_occupancy RPCs	Transaction, overlap, transfer, vacancy, and cross-tenant pgTAP scenarios
FR-OCC-003	Implemented	Tenancy and occupancy routes/services; create_tenancy_with_assignment and transfer_occupancy RPCs	Transaction, overlap, transfer, vacancy, and cross-tenant pgTAP scenarios
FR-OCC-004	Implemented	Tenancy and occupancy routes/services; create_tenancy_with_assignment and transfer_occupancy RPCs	Transaction, overlap, transfer, vacancy, and cross-tenant pgTAP scenarios
FR-OCC-005	Implemented	Tenancy and occupancy routes/services; create_tenancy_with_assignment and transfer_occupancy RPCs	Transaction, overlap, transfer, vacancy, and cross-tenant pgTAP scenarios
FR-OCC-006	Implemented	Tenancy and occupancy routes/services; create_tenancy_with_assignment and transfer_occupancy RPCs	Transaction, overlap, transfer, vacancy, and cross-tenant pgTAP scenarios
FR-OCC-007	Implemented	Tenancy and occupancy routes/services; create_tenancy_with_assignment and transfer_occupancy RPCs	Transaction, overlap, transfer, vacancy, and cross-tenant pgTAP scenarios
FR-BILL-001	Implemented	Billing service and routes; invoice/payment/allocation/receipt tables; generation and decision RPCs	Billing unit tests, idempotency and allocation pgTAP tests, owner/tenant end-to-end flow
FR-BILL-002	Implemented	Billing service and routes; invoice/payment/allocation/receipt tables; generation and decision RPCs	Billing unit tests, idempotency and allocation pgTAP tests, owner/tenant end-to-end flow
FR-BILL-003	Implemented	Billing service and routes; invoice/payment/allocation/receipt tables; generation and decision RPCs	Billing unit tests, idempotency and allocation pgTAP tests, owner/tenant end-to-end flow
FR-BILL-004	Implemented	Billing service and routes; invoice/payment/allocation/receipt tables; generation and decision RPCs	Billing unit tests, idempotency and allocation pgTAP tests, owner/tenant end-to-end flow
FR-BILL-005	Implemented	Billing service and routes; invoice/payment/allocation/receipt tables; generation and decision RPCs	Billing unit tests, idempotency and allocation pgTAP tests, owner/tenant end-to-end flow
FR-BILL-006	Implemented	Billing service and routes; invoice/payment/allocation/receipt tables; generation and decision RPCs	Billing unit tests, idempotency and allocation pgTAP tests, owner/tenant end-to-end flow
FR-BILL-007	Implemented	Billing service and routes; invoice/payment/allocation/receipt tables; generation and decision RPCs	Billing unit tests, idempotency and allocation pgTAP tests, owner/tenant end-to-end flow

Requirement	Status	Representative evidence	Verification method
FR-BILL-008	Implemented	Billing service and routes; invoice/payment/allocation/receipt tables; generation and decision RPCs	Billing unit tests, idempotency and allocation pgTAP tests, owner/tenant end-to-end flow
FR-BILL-009	Implemented	Billing service and routes; invoice/payment/allocation/receipt tables; generation and decision RPCs	Billing unit tests, idempotency and allocation pgTAP tests, owner/tenant end-to-end flow
FR-BILL-010	Implemented	Billing service and routes; invoice/payment/allocation/receipt tables; generation and decision RPCs	Billing unit tests, idempotency and allocation pgTAP tests, owner/tenant end-to-end flow
FR-BILL-011	Implemented	Billing service and routes; invoice/payment/allocation/receipt tables; generation and decision RPCs	Billing unit tests, idempotency and allocation pgTAP tests, owner/tenant end-to-end flow
FR-CMP-001	Implemented	Complaint routes and service; staff tasks; complaint/event schema and transition/assignment RPCs	Complaint service tests, lifecycle pgTAP tests, tenant-owner-staff smoke flow
FR-CMP-002	Implemented	Complaint routes and service; staff tasks; complaint/event schema and transition/assignment RPCs	Complaint service tests, lifecycle pgTAP tests, tenant-owner-staff smoke flow
FR-CMP-003	Implemented	Complaint routes and service; staff tasks; complaint/event schema and transition/assignment RPCs	Complaint service tests, lifecycle pgTAP tests, tenant-owner-staff smoke flow
FR-CMP-004	Implemented	Complaint routes and service; staff tasks; complaint/event schema and transition/assignment RPCs	Complaint service tests, lifecycle pgTAP tests, tenant-owner-staff smoke flow
FR-CMP-005	Implemented	Complaint routes and service; staff tasks; complaint/event schema and transition/assignment RPCs	Complaint service tests, lifecycle pgTAP tests, tenant-owner-staff smoke flow
FR-CMP-006	Implemented	Complaint routes and service; staff tasks; complaint/event schema and transition/assignment RPCs	Complaint service tests, lifecycle pgTAP tests, tenant-owner-staff smoke flow
FR-COM-001	Implemented	Notice service/routes; notifications and push registration; private upload helpers and storage policies	Targeting/RLS tests, upload limit checks, device tests with and without EAS configuration
FR-COM-002	Implemented	Notice service/routes; notifications and push registration; private upload helpers and storage policies	Targeting/RLS tests, upload limit checks, device tests with and without EAS configuration
FR-COM-003	Implemented	Notice service/routes; notifications and push registration; private upload helpers and storage policies	Targeting/RLS tests, upload limit checks, device tests with and without EAS configuration
FR-COM-004	Implemented	Notice service/routes; notifications and push registration; private upload helpers and storage policies	Targeting/RLS tests, upload limit checks, device tests with and without EAS configuration
FR-COM-005	Partially implemented	Notice service/routes; notifications and push registration; private upload helpers and storage policies	Targeting/RLS tests, upload limit checks, device tests with and without EAS configuration
FR-COM-006	Implemented	Notice service/routes; notifications and push registration; private upload helpers and storage policies	Targeting/RLS tests, upload limit checks, device tests with and without EAS configuration
FR-OPS-001	Implemented	Dashboard API; owner reports/operations routes; expense and agreement RPCs; CSV and receipt export	Aggregate/unit tests, report reconciliation, export inspection, database tests
FR-OPS-002	Implemented	Dashboard API; owner reports/operations routes; expense and agreement RPCs; CSV and receipt export	Aggregate/unit tests, report reconciliation, export inspection, database tests
FR-OPS-003	Implemented	Dashboard API; owner reports/operations routes; expense and agreement RPCs; CSV and receipt export	Aggregate/unit tests, report reconciliation, export inspection, database tests
FR-OPS-004	Implemented	Dashboard API; owner reports/operations routes; expense and agreement RPCs; CSV and receipt export	Aggregate/unit tests, report reconciliation, export inspection, database tests
FR-OPS-005	Implemented	Dashboard API; owner reports/operations routes; expense and agreement RPCs; CSV and receipt export	Aggregate/unit tests, report reconciliation, export inspection, database tests
NFR-SEC-001	Implemented	RLS and storage policies; auth/session configuration; environment rules	Positive/negative pgTAP tests, build/configuration review, secret scan
NFR-SEC-002	Implemented	RLS and storage policies; auth/session configuration; environment rules	Positive/negative pgTAP tests, build/configuration review, secret scan
NFR-SEC-003	Implemented	RLS and storage policies; auth/session configuration; environment rules	Positive/negative pgTAP tests, build/configuration review, secret scan
NFR-PRV-001	Implemented	Private storage policies and upload preparation	Unauthorized object tests, file validation tests, metadata review

Requirement	Status	Representative evidence	Verification method
NFR-PRV-002	Partially implemented	Private storage policies and upload preparation	Unauthorized object tests, file validation tests, metadata review
NFR-DAT-001	Implemented	Database schema, enums, constraints, triggers, transactional RPCs	Schema/constraint pgTAP tests and reconciliation cases
NFR-DAT-002	Implemented	Database schema, enums, constraints, triggers, transactional RPCs	Schema/constraint pgTAP tests and reconciliation cases
NFR-DAT-003	Implemented	Database schema, enums, constraints, triggers, transactional RPCs	Schema/constraint pgTAP tests and reconciliation cases
NFR-REL-001	Implemented	Transactional RPCs, configuration guard, release scripts/tests	Retry/failure tests, configuration smoke test, release checklist
NFR-REL-002	Implemented	Transactional RPCs, configuration guard, release scripts/tests	Retry/failure tests, configuration smoke test, release checklist
NFR-ERR-001	Implemented	AppError and to-user-message utilities; shared state views	Error translation unit tests and manual failure-state review
NFR-OFF-001	Partially implemented	Offline banner and online-only mutation behavior	Network-loss tests across read and protected mutation flows
NFR-ACC-001	Implemented	Shared components, design tokens, route screens	Screen reader, large-text, contrast, target-size, and reduced-motion review
NFR-ACC-002	Partially implemented	Shared components, design tokens, route screens	Screen reader, large-text, contrast, target-size, and reduced-motion review
NFR-CMP-001	Implemented	Expo/React Native configuration and EAS profiles	Android/iOS build and core-journey smoke tests
NFR-MNT-001	Implemented	TypeScript configuration, generated database types, feature modules, query keys, i18n	Typecheck, lint, format check, architecture review
NFR-PER-001	Partially implemented	Dashboard aggregate functions and scoped query keys	Large-dataset query and UI profiling
NFR-OBS-001	Partially implemented	Optional Sentry configuration and application initialization	Configured test event and sensitive-data inspection
NFR-REL-003	Partially implemented	Transactional RPCs, configuration guard, release scripts/tests	Retry/failure tests, configuration smoke test, release checklist

Appendix A. Roadmap and Deferred Scope

A.1 Implemented Extensions Beyond the Original P0

- Maintenance-staff role and assigned task workflow.
- Operational expense recording, voiding, and monthly profit summary.
- Agreement expiry and renewal workflow.

A.2 Planned Near-Term Requirements

Roadmap ID	Planned capability	Baseline treatment
RM-P1-001	Move-out request and financial settlement workflow	Planned; vacancy exists, but a tenant-requested settlement workflow is not baseline.
RM-P1-002	Scheduled rent generation and overdue reminders	Planned; current invoice generation is operator-initiated.
RM-P1-003	PDF owner reports	Planned; receipt PDF and CSV/report screens do not constitute a complete owner-report package.
RM-P1-004	QR tenant onboarding and receipt verification	Planned.
RM-P1-005	Persistent offline query cache and safe draft retry queue	Planned; current behavior is graceful degradation, not full offline synchronization.
RM-P1-006	Product analytics and feature flags	Planned.

A.3 Explicitly Excluded Modules

Exclusion	Reason for exclusion from version 1.0
Online payment gateway and subscription billing	The baseline prioritizes auditable manual payment tracking without provider complexity.
WhatsApp/SMS messaging	Communication remains in-app with optional push delivery.
Public marketplace, maps, discovery, and booking	Tenantly is a private property-operations product.
Platform-admin mobile console	Platform privileges require a separate trusted surface.
Visitor/security desk, electricity-meter, and mess modules	These operational domains are deferred until promoted by product planning.
Advanced forecasting, legal generation, ratings, and social features	These capabilities are not required for the current operational baseline.

Appendix B. Glossary

Term	Definition
Allocation	The portion of an approved payment applied to a specific invoice.
EAS	Expo Application Services used for supported build, update, and notification configuration.
Idempotent	Safe to repeat without creating duplicate business effects.
INR / paise	Indian rupees shown to users; one rupee equals 100 paise stored as an integer.
Membership	A user's role and status within an organization, optionally narrowed by property membership.
Occupancy assignment	A dated record linking a tenancy to a room and optionally a bed.
PG	Paying-guest accommodation.
RLS	PostgreSQL Row Level Security, which restricts rows according to authenticated identity and scope.
RPC	A protected database function invoked remotely by the authenticated client.
Signed URL	A time-limited authorized link used to retrieve a private stored object.
Supabase	The backend platform providing authentication, PostgreSQL, storage, and related services.

Term	Definition
Tenancy	The contractual and financial relationship between a resident and a property for a defined period.

Appendix C. References

1. Tenantly README.md - configuration, platform baseline, deployment, and verification commands.
2. Tenantly prd.md - implementation-ready product requirements and release priorities.
3. Tenantly DEMO.md - representative owner, tenant, and maintenance workflows.
4. Tenantly source routes and feature services under src/ - implemented client behavior and validation.
5. Tenantly Supabase migrations under supabase/migrations/ - schema, functions, constraints, triggers, RLS, and Storage policies.
6. Tenantly Jest and pgTAP tests under src/ and supabase/tests/ - verification assets and expected behavior.
7. IEEE/ISO/IEC 29148 software requirements engineering principles - structural inspiration; this document does not claim formal certification.

Appendix D. Requirement Status Legend

Status	Meaning
Implemented	Repository evidence supports the requirement in the current baseline.
Partially implemented	Core behavior exists, but configuration, coverage, or a documented quality gap remains.
Planned	Approved roadmap capability that is not part of the current baseline.
Excluded	Intentionally outside version 1.0.
External	Must be enforced outside the mobile client or by a future trusted platform surface.